

RETO 3 — EXAMEN FINAL

Documentación, Evaluación y Comunicación Arquitectónica

Sistema IoT · Detección de Fuga de Gas

Institución:

Corporación Universitaria Remington

Programa:

Ingeniería de Sistemas

Sede:

Montería, Córdoba

Docente:

Sonia Isabel Huérfano Duarte

Integrantes:

Tatiana Milena Arango Simanca

Luisa Fernanda Urueta Valencia

Fecha: 13/4/2026

Introducción

El presente documento corresponde al Reto 3, entrega final del curso de Arquitectura de Software. Sobre la base del Reto 1 (drivers y diseño inicial) y el Reto 2 (patrones y prototipo funcional), este reto sintetiza todo el trabajo en una documentación arquitectónica profesional usando el enfoque Views & Beyond, evalúa la arquitectura contra sus propios drivers mediante un checklist ATAM simplificado, y despliega el sistema completo con Docker Compose incluyendo health checks y monitoreo.

El sistema IoT de Detección de Fuga de Gas ha evolucionado a lo largo de las tres semanas: comenzó con la identificación de 8 drivers arquitectónicos y los diagramas C4 de contexto y contenedores; en el Reto 2 se profundizó con 3 patrones tácticos (Circuit Breaker, API Gateway y Repository) y el diagrama de componentes; y en este Reto 3 se documenta la arquitectura desde tres vistas complementarias, se evalúa su madurez y se conteneriza el stack completo para facilitar su despliegue reproducible.

Entregable 1: Documentación Arquitectónica — Views & Beyond

El enfoque Views & Beyond propone documentar la arquitectura desde múltiples perspectivas complementarias. Cada vista responde a las preguntas de un stakeholder diferente: la Vista de Módulos responde '¿cómo está dividido el código?', la Vista C&C responde '¿cómo se comunican los componentes en tiempo de ejecución?', y la Vista de Deployment responde '¿dónde se ejecuta el sistema?'. Juntas ofrecen una imagen completa de la arquitectura.

Vista 1: Módulos

Muestra la descomposición estática del sistema en unidades de código con responsabilidades bien definidas. Cada módulo expone una interfaz y declara sus dependencias, facilitando la mantenibilidad y el reemplazo independiente de partes.

Módulo	Responsabilidad	Depende de	Interfaz exportada
SensorReader	Lee niveles PPM del sensor MQ-2 cada 5 segundos y los entrega al CircuitBreaker.	Hardware (MQ-2/GPIO)	readSensor(): SensorData {ppm, timestamp}
CircuitBreaker	Monitorea la conexión MQTT. Abre el circuito tras 3 fallos; cierra al reconectar.	SensorReader	publish(data) / enqueue(data)
LocalQueue	Almacena lecturas en memoria SPIFFS cuando el circuito está abierto.	CircuitBreaker	enqueue(data) / drain(): SensorData[]
MQTTPublisher	Publica lecturas en tópicos MQTT con QoS 1 y autenticación TLS.	LocalQueue	publish(topic, payload)
AuthModule	Valida credenciales de clientes MQTT y aplica ACLs por tópico.	Ninguno	authenticate(client) / authorize(topic)
TopicRouter	Enruta mensajes entre publicadores y suscriptores según el tópico.	AuthModule	route(message, topic)
MQTTSubscriber	Suscribe a casa/gas/# y entrega mensajes al RuleEngine.	TopicRouter	onMessage(topic, payload)
RuleEngine	Evalúa si PPM supera el umbral. Dispara alertas y coordina persistencia.	MQTTSubscriber	evaluate(data): Alert null
DataRepository	Abstrae lectura/escritura en	RuleEngine	save(data) / query(filter): Reading[]

RETO 3 · EXAMEN FINAL			Sistema IoT - Detección de Fuga de Gas Corp. Universitaria Remington · 25% de la nota
Arquitectura de Software + IoT · Documentación y Evaluación			
Módulo	Responsabilidad	Depende de	Interfaz exportada
	InfluxDB. Desacopla la BD de la lógica.		
APIGateway	Expone datos via HTTP REST y WebSocket para el dashboard.	DataRepository	GET /api/ppm · WS /live
AlertNotifier	Envía notificación push/email cuando RuleEngine genera una alerta.	RuleEngine	notify(alert)
DashboardUI	Visualiza PPM en tiempo real, histórico y alertas activas.	APIGateway	UI: Gauge, Chart, AlertBanner

ADR-001 — Uso de MQTT sobre HTTP para telemetría IoT

Fecha: Semana 1 | Estado: Aceptado

Contexto: El ESP8266 tiene recursos limitados (80KB RAM, 80MHz). Se necesita transmitir lecturas cada 5 segundos con latencia mínima.

Decisión: Usar MQTT (Mosquitto 2.x) con QoS 1 y autenticación usuario/contraseña como protocolo de mensajería principal.

Consecuencias:  Latencia < 2s.  Bajo consumo de ancho de banda.  Patrón Pub-Sub nativo.  Mayor complejidad de configuración que HTTP.

Vista 2: Componentes y Conectores (C&C)

Muestra cómo se comunican los elementos del sistema en tiempo de ejecución: qué conectores usan, qué protocolo, y qué tipo de interacción establece cada relación. Esta vista es clave para entender el comportamiento dinámico del sistema IoT.

Conector / Relación	Tipo	Descripción y protocolo
ESP8266 → Mosquitto	Pipe (MQTT QoS 1)	El firmware publica PPM cada 5s en casa/gas/ppm. Con TLS en puerto 8883 en producción y 1883 en desarrollo. QoS 1 garantiza entrega al menos una vez.
Mosquitto → Node-RED	Pipe (MQTT interno)	Node-RED suscribe a casa/gas/# via nodo mqtt-in. El broker enruta mensajes internamente en la red Docker (iot_net).
Node-RED → InfluxDB	Llamada a procedimiento	El nodo influxdb-out escribe cada lectura como punto en la serie 'gas_readings'. Protocolo: HTTP/TCP interno en puerto 8086.
Node-RED → Grafana	Publicación de datos	Grafana consulta InfluxDB directamente via datasource configurado. Node-RED alimenta también via WebSocket para actualizaciones en tiempo real.
Node-RED → Telegram	Evento asíncrono	Al dispararse una alerta (PPM > 300), el nodo Telegram envía un mensaje HTTP POST a la Bot API de Telegram con el valor PPM y timestamp.
Grafana → Usuario	Cliente-Servidor (HTTP)	El usuario accede al dashboard Grafana vía navegador en puerto 3000. La comunicación es HTTP/WebSocket para actualizaciones en vivo.
HiveMQ Cloud → Mosquitto	MQTT Bridge (respaldo)	Configuración de bridge en Mosquitto para replicar mensajes hacia HiveMQ Cloud como fallback ante caída del broker local.

ADR-002 — Circuit Breaker con almacenamiento local en SPIFFS

Fecha: Semana 2 | Estado: Aceptado

Contexto: El driver de disponibilidad 24/7 exige 0% de pérdida de lecturas. El ESP8266 puede perder conectividad en zonas con WiFi inestable.

Decisión: Implementar Circuit Breaker en firmware: tras 3 fallos MQTT, almacenar en SPIFFS y reintentar cada 30s. Al reconectar, drena la cola.

Consecuencias:  0 lecturas perdidas en desconexiones < 10 min.  Ahorro de batería.  Ráfaga de mensajes al reconectar (QoS 1 la mitiga).

Vista 3: Deployment (Despliegue)

Muestra dónde se ejecuta cada artefacto del sistema: qué nodo físico o virtual lo aloja, qué puerto expone y cómo se relaciona con los demás nodos en la infraestructura real. Esta vista es la base para el Docker Compose del Entregable 3.

Nodo de despliegue	Artefacto	Puerto	Descripción
Dispositivo IoT (ESP8266)	Firmware MicroPython	GPIO / WiFi	Microcontrolador con sensor MQ-2. Se conecta a la red WiFi doméstica y publica en el broker MQTT. En simulación: script Python en Docker.
Servidor local (Docker)	eclipse-mosquitto:2.0	1883 / 9001	Broker MQTT contenerizado. Configurado con autenticación por usuario/contraseña y persistencia en volumen Docker.
Servidor local (Docker)	nodered/node-red:3.1	1880	Motor de procesamiento. Flujos configurados para suscripción MQTT, evaluación de umbrales, persistencia y notificaciones.
Servidor local (Docker)	influxdb:2.7	8086	Base de datos de series temporales. Almacena lecturas PPM con timestamp. Org: <code>iot_gas</code> , Bucket: <code>gas_readings</code> .
Servidor local (Docker)	grafana/grafana:10.2	3000	Dashboard de monitoreo. Datasource configurado contra InfluxDB. Panel de PPM en tiempo real e histórico de alertas.
Cloud (externo)	HiveMQ Cloud	8883 (TLS)	Broker MQTT externo como respaldo. El Mosquitto local lo configura como bridge para replicar mensajes en caso de necesidad.
Cloud (externo)	Telegram Bot API	443 (HTTPS)	Servicio de notificaciones externo. Node-RED se comunica via HTTP POST para enviar alertas push al usuario.

ADR-003 — Node-RED como capa de procesamiento y Gateway

Fecha: Semana 2 | Estado: Aceptado

- Contexto:** El equipo necesita implementar lógica de umbral, persistencia y notificaciones en tiempo académico limitado sin un backend personalizado.
- Decisión:** Usar Node-RED como motor de procesamiento visual, API Gateway y orquestador de notificaciones contenerizado en Docker.
- Consecuencias:**  Desarrollo visual sin compilar.  Integración nativa MQTT/InfluxDB/Telegram.  Menor control para lógica compleja.  No escala bien a >100 sensores.

Entregable 2: Evaluación de Arquitectura — Checklist ATAM Simplificado

El método ATAM (Architecture Tradeoff Analysis Method) evalúa si la arquitectura satisface los atributos de calidad definidos como drivers. Esta versión simplificada toma cada driver del Reto 1 y lo evalúa contra el diseño del Reto 2, identificando evidencias, riesgos y puntos sensibles. La honestidad en esta evaluación es más valiosa que reportar todo como 'cumplido'.

Driver / Escenario del Reto 1	¿Cumple?	Evidencia en la arquitectura	Riesgo identificado	Punto sensible / Trade-off
Latencia < 10 s: alerta debe llegar al usuario en menos de 10 s tras detección	✓ Sí	MQTT QoS 1 + Node-RED en red local Docker. Latencia medida en pruebas: ~2-4 s sensor → Telegram.	En picos de red WiFi doméstica puede alcanzar 8-9 s. Dentro del límite pero con margen ajustado.	Trade-off: QoS 1 garantiza entrega pero duplica mensajes en reconexión. Punto sensible: la latencia de Telegram depende de la red externa.
Disponibilidad 24/7: el sistema no debe perder lecturas críticas	⚠ Parcial	Circuit Breaker + LocalQueue en SPIFFS. Recuperación automática de conexión en < 2 minutos.	Sin UPS o batería de respaldo, el dispositivo IoT se apaga en corte eléctrico. Riesgo alto en detección nocturna.	Punto sensible: el Circuit Breaker protege contra fallos de red, no contra fallos de energía. Se recomienda batería de respaldo para producción.
Seguridad (Integridad): 100% de publicaciones no autorizadas bloqueadas	⚠ Parcial	AuthModule con usuario/contraseña en Mosquitto. ACLs por tópico configuradas.	Sin TLS en desarrollo (puerto 1883). Los datos viajan en texto plano en la red local. En producción se requiere puerto 8883 con certificados.	Trade-off: facilidad de desarrollo sin TLS vs. seguridad en producción. Punto sensible: si el archivo passwd se expone, toda la autenticación colapsa.
Hardware de bajo consumo: sensores y MCU con recursos limitados	✓ Sí	ESP8266 (80MHz, 80KB RAM). Firmware optimizado: publica solo cambios significativos (>5 PPM de diferencia).	El CircuitBreaker con SPIFFS tiene un límite de ~1MB de almacenamiento local. En desconexiones largas se pierden lecturas antiguas.	Trade-off: frecuencia de publicación cada 5s vs. consumo de batería. Aumentar a cada 60s reduciría consumo un 80% pero perdería resolución temporal.
Uso de protocolos estándar IoT (MQTT): interoperabilidad garantizada	✓ Sí	Mosquitto 2.x implementa MQTT 3.1.1 y 5.0. Bridge configurado con HiveMQ Cloud. Node-RED usa nodos MQTT estándar.	La versión MQTT 5.0 no es soportada por todas las librerías de ESP8266 (PubSubClient solo soporta 3.1.1).	Punto sensible: la incompatibilidad de versiones MQTT entre ESP8266 (3.1.1) y broker (5.0) puede causar problemas en features avanzados como session expiry.
Detección y publicación MQTT: sensor publica PPM cada 5 s en tópico	✓ Sí	Simulador Python publica en casa/gas/ppm cada 5s con payload JSON {ppm,	En hardware real, el sensor MQ-2 requiere	Trade-off: precisión del sensor MQ-2 (económico, ~\$2 USD) vs. sensores industriales (>\$50)

RETO 3 · EXAMEN FINAL				Sistema IoT - Detección de Fuga de Gas Corp. Universitaria Remington · 25% de la nota
Arquitectura de Software + IoT · Documentación y Evaluación				
Driver / Escenario del Reto 1	¿Cumple?	Evidencia en la arquitectura	Riesgo identificado	Punto sensible / Trade-off
específico		timestamp, sensor}. Verificado en pruebas con mosquito_sub.	calentamiento de 24-48h para lecturas precisas. Las primeras lecturas pueden ser imprecisas.	USD). Para contexto académico es suficiente.
Generación de alerta crítica: notificación en < 10 s cuando PPM > 300	✓ Sí	RuleEngine en Node-RED evalúa umbral en cada mensaje. AlertNotifier envía Telegram en < 1s adicional tras la detección.	El umbral de 300 PPM es configurable pero está hardcodeado en el flujo Node-RED. Cambios requieren editar el flujo manualmente.	Punto de mejora: externalizar la configuración del umbral a una variable de entorno o panel de administración para mayor mantenibilidad.

💡 **Nota:** Leyenda: ✓ Sí = el driver se cumple con evidencia técnica concreta. ⚠ Parcial = se cumple con limitaciones conocidas.
 ✗ No = no se cumple en la arquitectura actual.

Entregable 3: Despliegue con Docker Compose + Monitoreo

El archivo docker-compose.yml conteneriza los cuatro servicios principales del sistema IoT: Mosquitto (broker MQTT), Node-RED (procesamiento), InfluxDB (persistencia) y Grafana (visualización). Todos los servicios están en la red interna `iot_net`, con health checks configurados y políticas de reinicio automático.

Servicio	Imagen Docker	Puerto	Health Check configurado
mosquitto	eclipse-mosquitto:2.0	1883/9001	mosquitto_sub -t \$\$\$SYS/# -C 1 — Verifica que el broker acepta conexiones. Interval: 30s, timeout: 10s, retries: 3
nodered	nodered/node-red:3.1	1880	wget http://localhost:1880/ — Verifica que la UI de Node-RED responde. Depende de mosquitto e influxdb healthy.
influxdb	influxdb:2.7	8086	influx ping — Comando nativo de InfluxDB. Verifica que la API HTTP está activa. start_period: 30s para inicialización.
grafana	grafana/grafana:10.2	3000	wget http://localhost:3000/api/health — Endpoint de health nativo de Grafana. Depende de influxdb healthy.
sensor_simulator	python:3.11-slim	—	No requiere health check. Servicio opcional; se puede quitar en despliegue con hardware real (ESP8266 físico).

Comandos de operación

```
# Levantar todos los servicios en background
docker compose up -d

# Verificar que todos están healthy
docker compose ps

# Ver logs en tiempo real
docker compose logs -f

# Ver logs de un servicio específico
docker compose logs -f mosquitto

# Reiniciar un servicio sin bajar los demás
docker compose restart nodered

# Inspeccionar health check de un contenedor
docker inspect iot_mosquitto | grep -A5 '"Health"'

# Detener y limpiar todo (incluyendo volúmenes)
docker compose down -v
```



Nota: El archivo `docker-compose.yml` completo y el `sensor_sim.py` están incluidos en la entrega del repositorio GitHub.

Entregable 4: Portafolio GitHub

El repositorio GitHub incluye el README profesional con descripción del proyecto, diagramas C4 en formato Mermaid (renderizados automáticamente por GitHub), las 3 ADRs documentadas, instrucciones de instalación paso a paso, comandos de prueba MQTT y el enlace al video demo. La estructura de carpetas refleja la Vista de Deployment.

RETO 3 · EXAMEN FINAL		Sistema IoT - Detección de Fuga de Gas Corp. Universitaria Remington · 25% de la nota
Arquitectura de Software + IoT · Documentación y Evaluación		
Sección del README	Contenido	
Descripción + badges	Resumen del proyecto en 2-3 líneas. Badges de Docker, MQTT, Node-RED, InfluxDB y Grafana.	
Arquitectura (C4)	Diagramas Mermaid de los 3 niveles C4 renderizados directamente en el README de GitHub.	
Requisitos + instalación	Docker >= 24.x, Docker Compose >= 2.x. Pasos numerados desde git clone hasta docker compose up.	
Tabla de servicios	URL de acceso, usuario y contraseña de cada servicio del stack.	
Estructura del proyecto	Árbol de directorios del repositorio explicado.	
ADRs (3 decisiones)	ADR-001 MQTT, ADR-002 Circuit Breaker, ADR-003 Node-RED. Formato: Contexto + Decisión + Consecuencias.	
Demo MQTT (comandos)	Comandos docker exec para publicar/suscribir mensajes de prueba sin instalar nada extra.	
Equipo + referencias	Nombres de integrantes y 5 referencias académicas en formato APA.	

💡 **Nota:** El README.md está disponible como archivo separado en la entrega. Copiarlo a la raíz del repositorio GitHub.

Entregable 5: Demo Day — Presentación 10-12 min + Poster/Video 2 min

La presentación final defiende la arquitectura con evidencia técnica ante el curso. No es suficiente mostrar que funciona: hay que explicar por qué se tomaron las decisiones y qué se sacrificó en cada trade-off. La estructura minuto a minuto asegura cubrir todos los entregables en el tiempo disponible.

Minuto	Sección	Contenido a demostrar
0:00–1:00	Introducción	Nombre del equipo, proyecto y problema. Mostrar foto del sensor MQ-2 / screenshot de Wokwi. Contexto: ¿por qué es importante detectar fugas de gas?
1:00–2:30	Reto 1 recap	Mostrar los 3 diagramas C4 (contexto, contenedores, componentes). Resaltar los 3 drivers más críticos: latencia, disponibilidad y seguridad.
2:30–4:30	Patrones aplicados	Explicar Circuit Breaker, Gateway y Repository con el diagrama de componentes. Mostrar el código/flujo Node-RED en pantalla.
4:30–7:00	Demo en vivo	Levantar docker compose up -d. Mostrar los 4 contenedores healthy. Publicar PPM normal → ver en Grafana. Forzar PPM > 300 → mostrar alerta en Telegram.
7:00–8:30	Views & Beyond	Mostrar la Vista de Módulos y la Vista de Deployment. Explicar cómo el Docker Compose implementa la vista de deployment.
8:30–10:00	ATAM y trade-offs	Mostrar el checklist ATAM. Destacar el riesgo de disponibilidad sin UPS y el trade-off de TLS. Demostrar honestidad arquitectónica.
10:00–11:00	GitHub y ADRs	Mostrar el README del repositorio con los 3 ADRs. Señalar los diagramas Mermaid renderizados en GitHub.
11:00–12:00	Conclusiones	¿Qué aprendimos? ¿Qué haríamos diferente en producción? Preguntas del curso. Link al video y al repositorio GitHub.

Poster / Video resumen (2 minutos)

El poster o video de 2 minutos debe resumir: (1) el problema que resuelve el sistema, (2) la arquitectura en una imagen (C4 Nivel 2), (3) los 3 patrones aplicados con una frase cada uno, y (4) el resultado: sistema funcionando con alerta real en Telegram. Puede ser una grabación de pantalla con voz en off o un poster impreso en A2.

 **Nota:** Subir el video a YouTube (no listado) o Google Drive y colocar el enlace en el README del repositorio GitHub antes de la presentación.

Referencias

Referencias bibliográficas en formato APA 7.^a edición consultadas para el desarrollo de los tres retos del curso.

- [1] Bass, L., Clements, P., & Kazman, R. (2022). Software architecture in practice (4.^a ed.). Addison-Wesley Professional.
- [2] Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., ... Stafford, J. (2010). Documenting software architectures: Views and beyond (2.^a ed.). Addison-Wesley. <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=30386>
- [3] Kazman, R., Klein, M., & Clements, P. (2000). ATAM: Method for architecture evaluation. Software Engineering Institute, Carnegie Mellon University. <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=5177>
- [4] Brown, S. (2022). The C4 model for visualising software architecture. Recuperado de <https://c4model.com>
- [5] Nygard, M. T. (2018). Release it! Design and deploy production-ready software (2.^a ed.). Pragmatic Bookshelf.
- [6] Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley. Recuperado de <https://martinfowler.com/eaCatalog/>
- [7] Eclipse Foundation. (2023). Eclipse Mosquitto: An open source MQTT broker (v2.x). Documentación oficial. Recuperado de <https://mosquitto.org/documentation/>
- [8] OpenJS Foundation. (2023). Node-RED: Low-code programming for event-driven applications. Documentación oficial. Recuperado de <https://nodered.org/docs/>
- [9] InfluxData. (2023). InfluxDB 2.x documentation. Recuperado de <https://docs.influxdata.com>
- [10] OASIS. (2019). MQTT version 5.0. OASIS Standard. Recuperado de <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>